

# **CMPUT 267: Machine Learning I · Fall 2026**

# **Gradient Descent**

Xi Ye

2026-10-01

# Recall: Closed Form Linear Regression

For squared-loss linear regression,

$$\hat{L}_{\mathcal{D}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i^{\top} \mathbf{w} - y_i)^2.$$

The previous lecture found the exact minimizer by solving

$$A\hat{\mathbf{w}} = \mathbf{b}, \quad \hat{\mathbf{w}} = A^{-1}\mathbf{b} \quad \text{when } A \text{ is invertible.}$$



The closed form works because this particular function class and loss produce a solvable system.

# This Lecture: (First-order) Gradient Descent

We will introduce another optimization method: first-order gradient descent

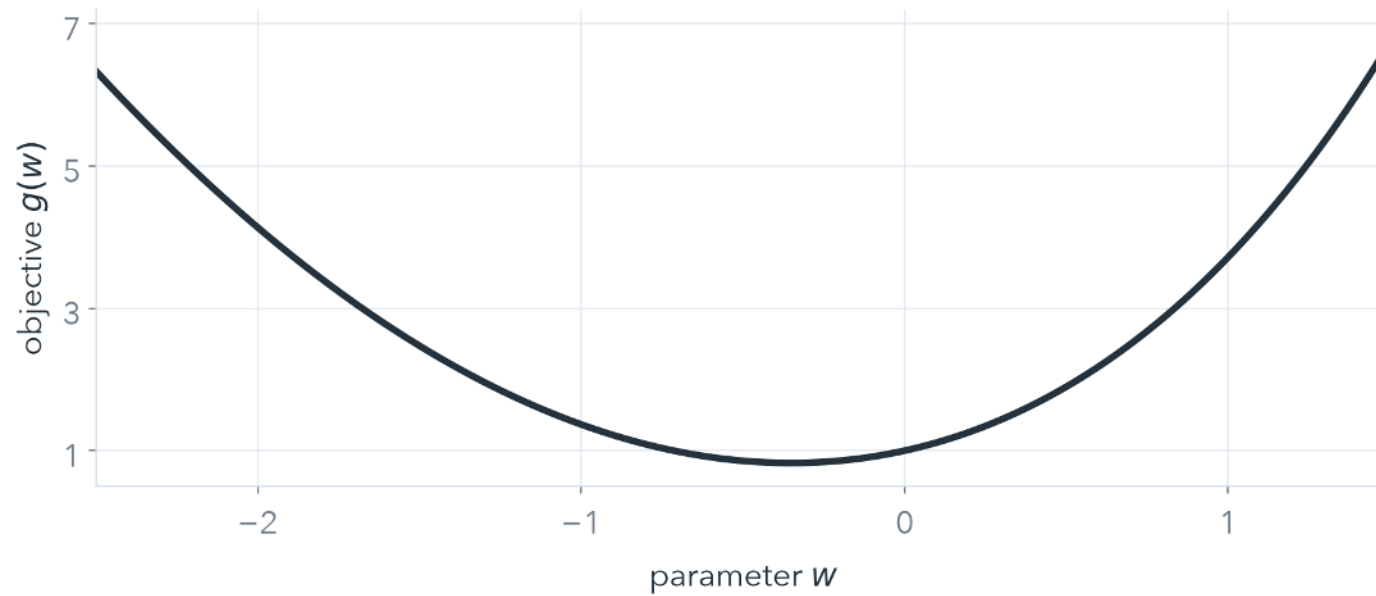
## Why another one?

- for linear regression,  $A^{-1}$  is very expensive.  $\Omega(n^{2.373})$  complexity.
- for convex functions beyond linear regression,  $g'(w) = 0$  does not always have a simple closed form solution

# A Convex Objective

Consider

$$g(w) = w^2 + e^w.$$



? Where does this function attain its minimum?

# A Convex Objective Without an Easy Solution

Is it convex?

$$g'(w) = 2w + e^w, \quad g''(w) = 2 + e^w > 0.$$

The objective is convex, so a solution of

$$2w + e^w = 0$$

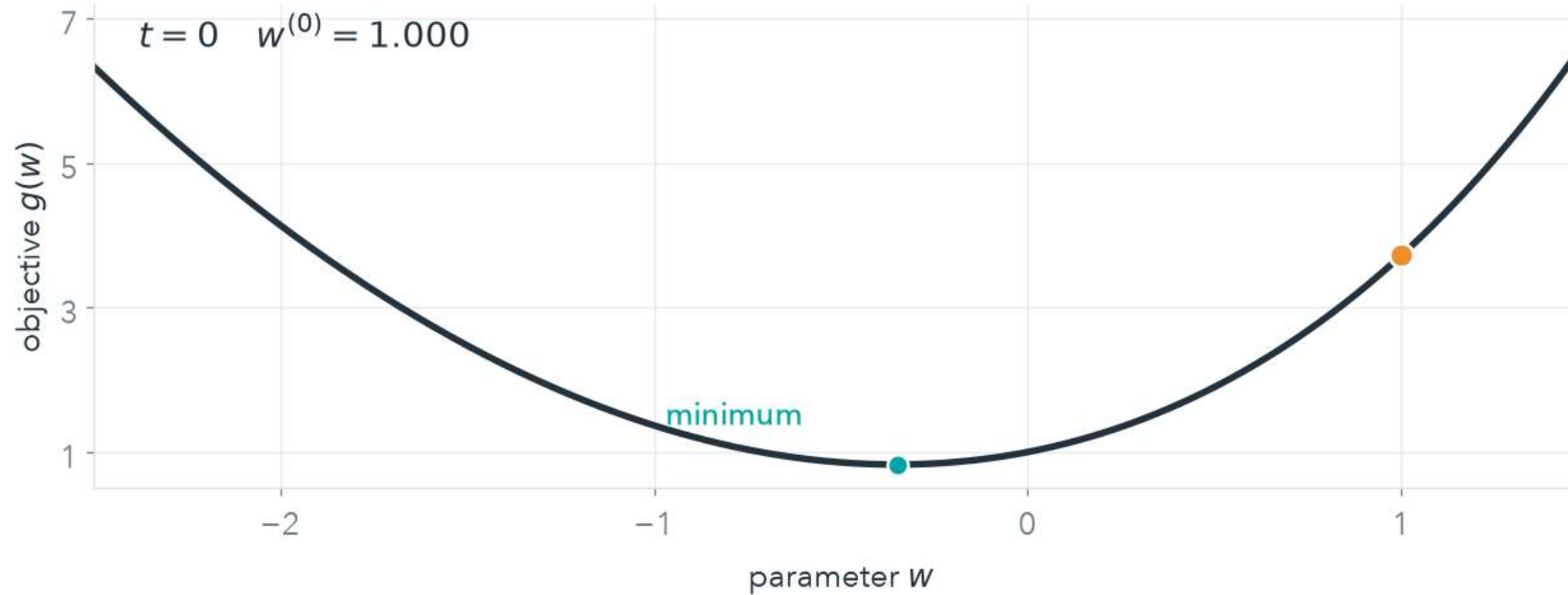
would be its unique minimizer.

  $2w = -e^w$  does not have a closed form solution.

# SEARCH

# A First Look at Gradient Descent

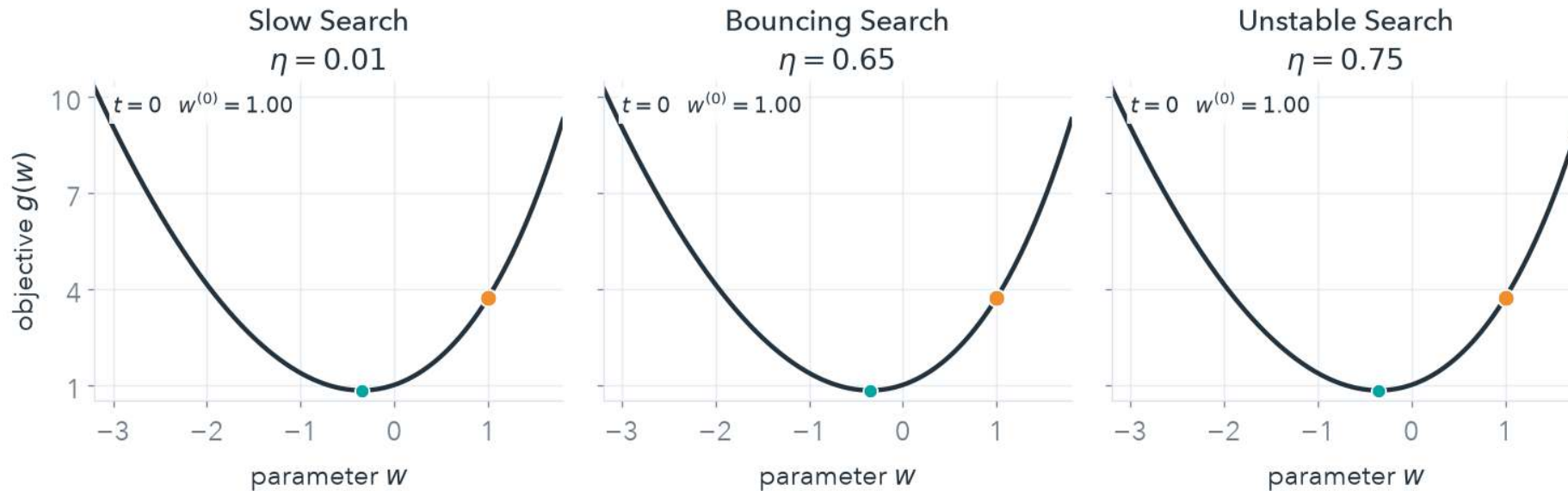
Start from  $w^{(0)} = 1.0$  and repeatedly use the local slope to move downhill.



*i* Each step improves the current guess. The search gradually approaches the minimum.

# Three Searches, One Objective

Same objective and starting point  $w^{(0)} = 1.0$ . Search in different ways.



# Optimization via Iterative Search

Gradient descent: iteratively approaching the minimal, based on slope:

$$w^{(0)}, w^{(1)}, w^{(2)}, \dots$$

## Current Guess

$$w^{(t)}$$

The parameter available at iteration  $t$ .

## Update

$$w^{(t)} \mapsto w^{(t+1)}$$

A small step intended to reduce the objective.



We use derivatives (gradients in the multivariate case) to determine the direction of the step.

# A One-Dimensional Example

Continue with the same objective:

$$g(w) = w^2 + e^w.$$

## Derivative

$$g'(w) = 2w + e^w$$

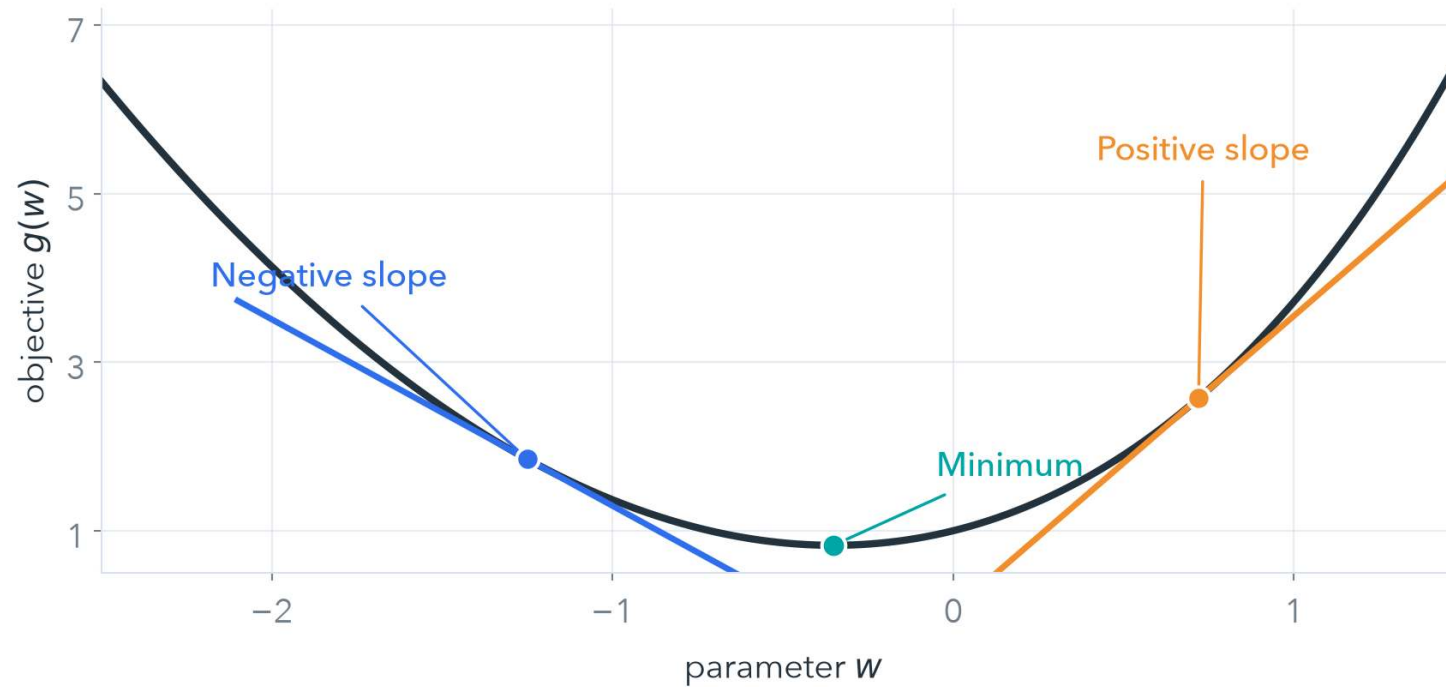
The local slope at  $w$ .

## Minimizer

$$w^* \approx -0.352$$

A numerical target for checking the search.

# The Derivative Gives a Direction



*i* A positive derivative means smaller values lie locally to the left. A negative derivative means smaller values lie locally to the right.

# The Gradient Descent Update

## Gradient Descent

Starting from an initial guess, repeatedly move in the negative-gradient direction.

NEXT POINT

$$w^{(t+1)}$$

=

CURRENT POINT

$$w^{(t)}$$

-

STEP

$$\eta^{(t)} g'(w^{(t)})$$

  $\eta^{(t)} > 0$  is the **learning rate**, also called the **step size**.

# The Negative Derivative Points Downhill

For a small change  $\Delta w$ , the derivative gives the local approximation

$$g(w + \Delta w) \approx g(w) + g'(w)\Delta w.$$

Choose  $\Delta w = -\eta g'(w)$  with  $\eta > 0$ :

$$g(w - \eta g'(w)) \approx g(w) - \eta (g'(w))^2.$$

# One Gradient Descent Step

Let

$$g(w) = w^2 + e^w, \quad w^{(0)} = 1, \quad \eta = 0.12.$$

$$g'(w^{(0)}) = 2 + e \approx 4.718,$$
$$w^{(1)} = 1 - 0.12(4.718) \approx 0.434.$$

 The positive derivative moves the parameter left, toward the minimizer near  $-0.352$ .

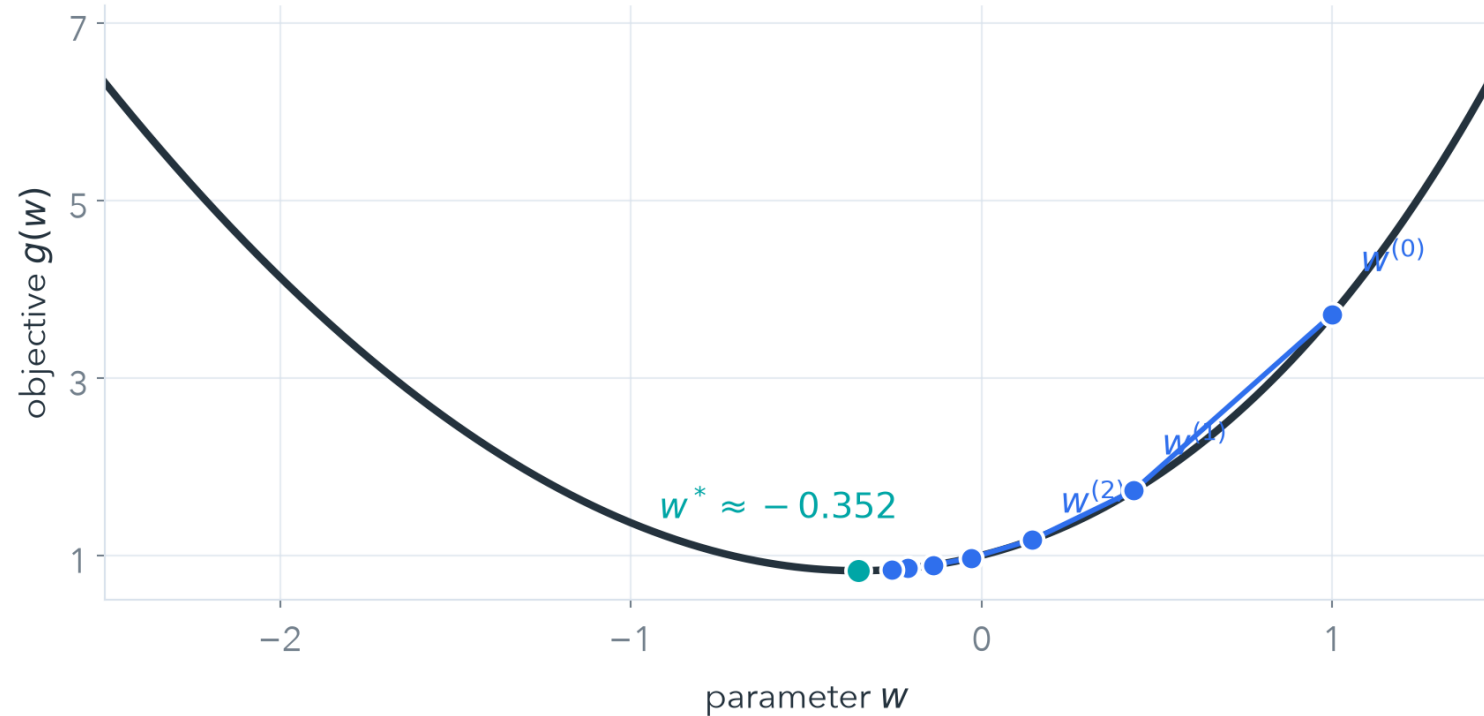
# Repeating the Update

Using the same objective and  $\eta = 0.12$ :

| Iteration $t$ | Parameter $w^{(t)}$ | Derivative $g'(w^{(t)})$ | Objective $g(w^{(t)})$ |
|---------------|---------------------|--------------------------|------------------------|
| 0             | 1.000               | 4.718                    | 3.718                  |
| 1             | 0.434               | 2.411                    | 1.731                  |
| 2             | 0.145               | 1.445                    | 1.176                  |
| 3             | -0.029              | 0.914                    | 0.972                  |

 The parameter approaches  $w^* \approx -0.352$ , while the objective approaches  $g(w^*) \approx 0.827$ .

# Gradient Descent as a Trajectory



*i* The derivative becomes smaller near the minimizer, so a fixed learning rate naturally produces shorter steps on this objective.

# The Basic Algorithm

**Input:** differentiable objective  $g$ , learning rate  $\eta$ , number of iterations  $T$

1. Choose an initial parameter  $w^{(0)}$ .
2. For  $t = 0, \dots, T - 1$ :
  - compute the derivative  $g'(w^{(t)})$ ;
  - set  $w^{(t+1)} = w^{(t)} - \eta g'(w^{(t)})$ .
3. Return  $w^{(T)}$  as an approximate minimizer.

 Unlike a closed form, the returned answer depends on the initialization, learning rate, and number of iterations.

# When Should the Algorithm Stop?

We monitor the objective after each update.

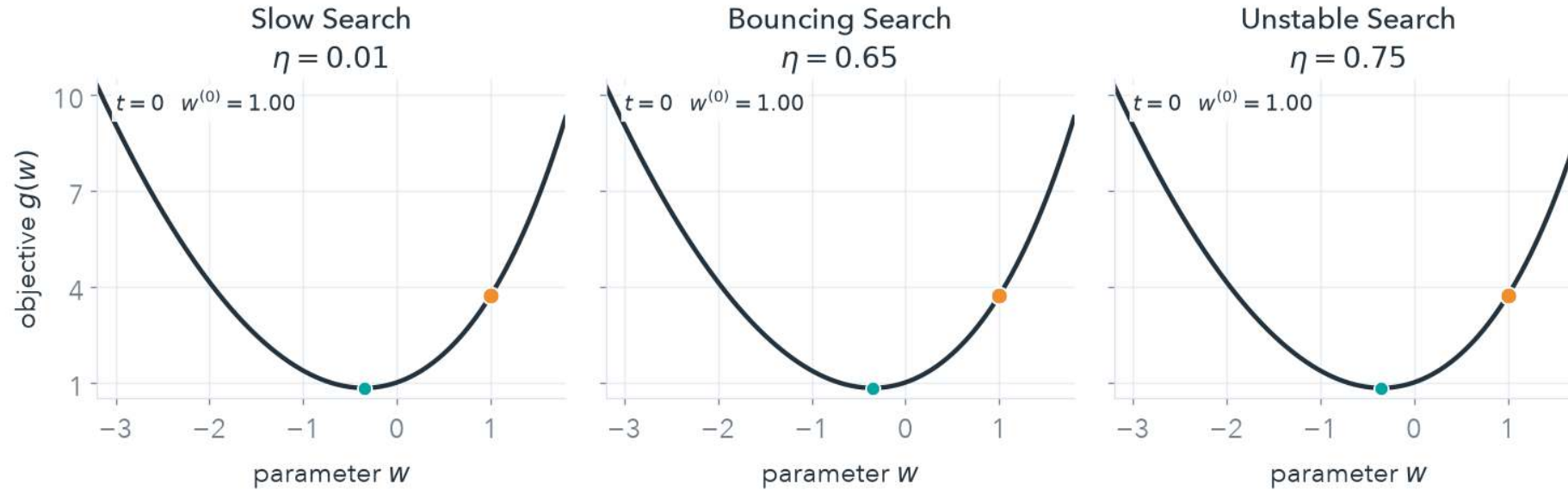
$$\Delta^{(t)} = \left| g(w^{(t+1)}) - g(w^{(t)}) \right|.$$



Stop when  $\Delta^{(t)} \leq \epsilon$ .

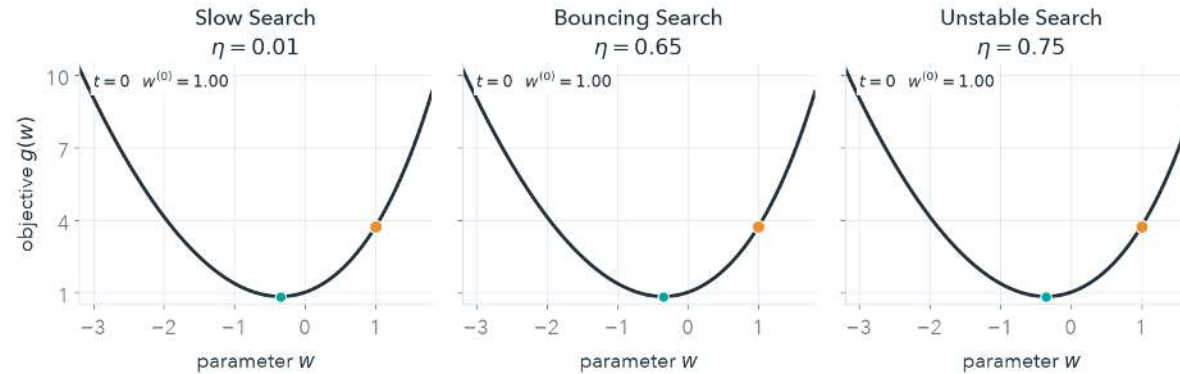
$\epsilon > 0$  is a small tolerance. A smaller  $\epsilon$  requires the objective to change less before the search stops.

# The Learning Rate Changes the Path



*i* The same objective and initialization can produce slow convergence, useful progress, or divergence solely because  $\eta$  changes.

# Learning-Rate Tradeoffs



## Smaller learning rate

- Shorter, safer updates
- More iterations to reach a useful solution
- Less likely to jump across a narrow minimum

## Larger learning rate

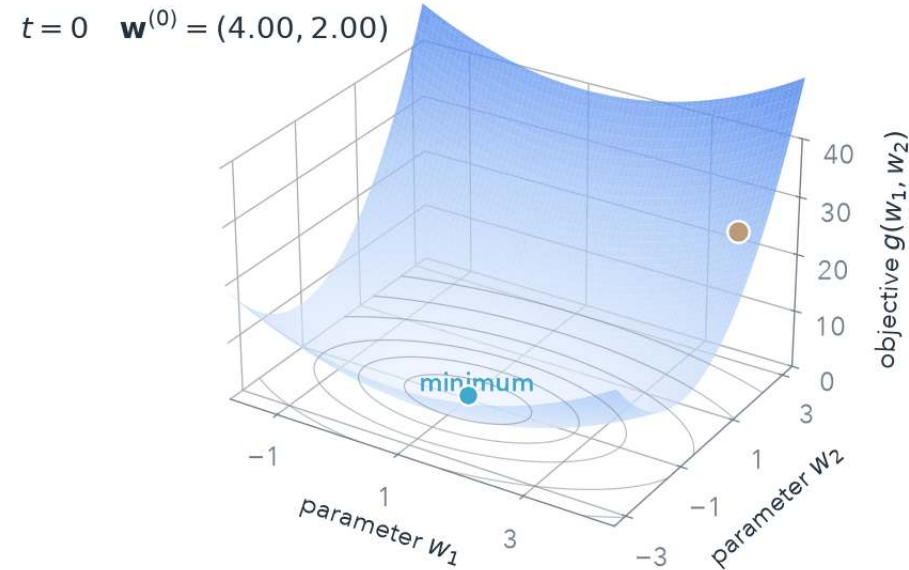
- Faster movement when the scale is appropriate
- Can cross the minimum repeatedly
- Can increase the objective and diverge

? If the objective increases rapidly for several updates, which hyperparameter should you inspect first?

# Gradient Descent with Two Parameters

Consider

$$g(w_1, w_2) = (w_1 - 1)^2 + 2(w_2 + 1)^2.$$



# Multivariate Gradient Descent

Machine-learning predictors usually depend on a vector

$$\mathbf{w} = (w_1, \dots, w_d)^\top.$$

 We need a direction in the parameter space, not merely “left” or “right.”

# The Gradient Vector

## Gradient

The vector containing the partial derivative of the objective with respect to every parameter.

$$\nabla g(\mathbf{w}) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(\mathbf{w}) \\ \vdots \\ \frac{\partial g}{\partial w_d}(\mathbf{w}) \end{bmatrix}.$$



The gradient points in the direction of the greatest local increase. Its negative points in the direction of the greatest local decrease.

# Multivariate Gradient Descent

The scalar update extends by replacing the derivative with the gradient:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta^{(t)} \nabla g(\mathbf{w}^{(t)})$$

Each coordinate updates simultaneously:

$$w_j^{(t+1)} = w_j^{(t)} - \eta^{(t)} \frac{\partial g}{\partial w_j}(\mathbf{w}^{(t)}).$$

# A Two-Dimensional Objective

Consider

$$g(w_1, w_2) = (w_1 - 1)^2 + 2(w_2 + 1)^2.$$

Its gradient is

$$\nabla g(w_1, w_2) = \begin{bmatrix} 2(w_1 - 1) \\ 4(w_2 + 1) \end{bmatrix}.$$

# One Multivariate Update

Recall the objective and its gradient:

$$g(w_1, w_2) = (w_1 - 1)^2 + 2(w_2 + 1)^2, \quad \nabla g(w_1, w_2) = \begin{bmatrix} 2(w_1 - 1) \\ 4(w_2 + 1) \end{bmatrix}.$$

Start from  $\mathbf{w}^{(0)} = (4, 2)^\top$  with  $\eta = 0.18$ .

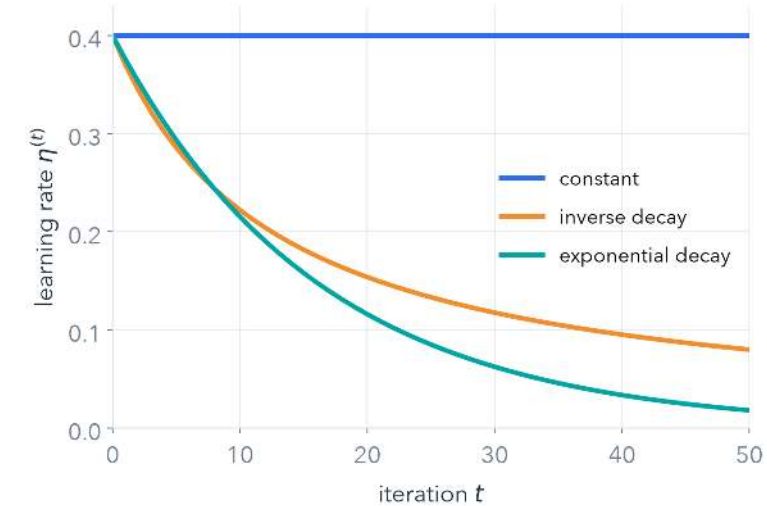
$$g(4, 2) = 27, \quad \nabla g(\mathbf{w}^{(0)}) = \begin{bmatrix} 6 \\ 12 \end{bmatrix} \quad \mathbf{w}^{(1)} = \begin{bmatrix} 4 \\ 2 \end{bmatrix} - 0.18 \begin{bmatrix} 6 \\ 12 \end{bmatrix} = \begin{bmatrix} 2.92 \\ -0.16 \end{bmatrix}.$$

*i* The next step starts from  $\mathbf{w}^{(1)}$ , where  $g(2.92, -0.16) = 5.0976 < 27$ .

# Learning-Rate Schedules

The learning rate may stay fixed or decrease with the iteration  $t$ .

- **Constant:**  $\eta^{(t)} = \eta_0$
- **Inverse decay:**  $\eta^{(t)} = \frac{\eta_0}{1 + \lambda t}$
- **Exponential decay:**  $\eta^{(t)} = \eta_0 \rho^t$ , where  $0 < \rho < 1$



Decay permits larger steps early and smaller steps later. The constants  $\lambda$  and  $\rho$  control how quickly the learning rate shrinks.

# Normalized Gradient Descent

Use the gradient for direction while removing its magnitude:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta^{(t)} \frac{\nabla g(\mathbf{w}^{(t)})}{\|\nabla g(\mathbf{w}^{(t)})\|}, \quad \nabla g(\mathbf{w}^{(t)}) \neq \mathbf{0}.$$

## Cost Parameter-step Length

$$\|\Delta \mathbf{w}^{(t)}\| = \eta^{(t)}.$$

# Gradient Descent for Linear Regression

# Minimizing Empirical Risk

For a parameterized predictor  $f_{\mathbf{w}}$ , define

$$\hat{L}_{\mathcal{D}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{w}}(\mathbf{x}_i), y_i).$$

Gradient descent can minimize this objective:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta^{(t)} \nabla \hat{L}_{\mathcal{D}}(\mathbf{w}^{(t)}).$$



Optimization changes the parameters. The dataset, loss, and function class define the objective being optimized.

# Squared-Loss Linear Regression

For

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^\top \mathbf{w}, \quad \ell(\hat{y}, y) = (\hat{y} - y)^2,$$

the empirical risk is

$$\hat{L}_{\mathcal{D}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i^\top \mathbf{w} - y_i)^2.$$

# The Gradient of One Example Loss

Let

$$L_i(\mathbf{w}) = (\mathbf{x}_i^\top \mathbf{w} - y_i)^2.$$

For coordinate  $w_j$ , the chain rule gives

$$\frac{\partial L_i}{\partial w_j}(\mathbf{w}) = 2 (\mathbf{x}_i^\top \mathbf{w} - y_i) \frac{\partial}{\partial w_j} (\mathbf{x}_i^\top \mathbf{w} - y_i) = 2 (\mathbf{x}_i^\top \mathbf{w} - y_i) x_{ij}.$$

# One Example Contributes a Gradient Vector

Stack means placing one partial derivative per parameter into a column vector:

$$\nabla L_i(\mathbf{w}) = \begin{bmatrix} \frac{\partial L_i}{\partial w_0}(\mathbf{w}) \\ \frac{\partial L_i}{\partial w_1}(\mathbf{w}) \\ \vdots \\ \frac{\partial L_i}{\partial w_d}(\mathbf{w}) \end{bmatrix} = 2 (\mathbf{x}_i^\top \mathbf{w} - y_i) \begin{bmatrix} x_{i0} \\ x_{i1} \\ \vdots \\ x_{id} \end{bmatrix} = 2 (\mathbf{x}_i^\top \mathbf{w} - y_i) \mathbf{x}_i.$$

# The Linear Regression Gradient

The gradient distributes across the average:

$$\nabla \hat{L}_{\mathcal{D}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \nabla L_i(\mathbf{w}) = \frac{2}{n} \sum_{i=1}^n (\mathbf{x}_i^\top \mathbf{w} - y_i) \mathbf{x}_i.$$

# Batch Gradient Descent

Substitute the linear-regression gradient into the update:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta^{(t)} \frac{2}{n} \sum_{i=1}^n \left( \mathbf{x}_i^\top \mathbf{w}^{(t)} - y_i \right) \mathbf{x}_i$$

## Batch Gradient Descent

The batch contains the **ENTIRE dataset** of  $n$  training examples, and every update uses that **FULL BATCH**.

# A Batch Gradient Descent Learner

**Input:** dataset  $\mathcal{D}$ , learning rate  $\eta$ , number of updates  $T$

1. Initialize  $\mathbf{w}$ .

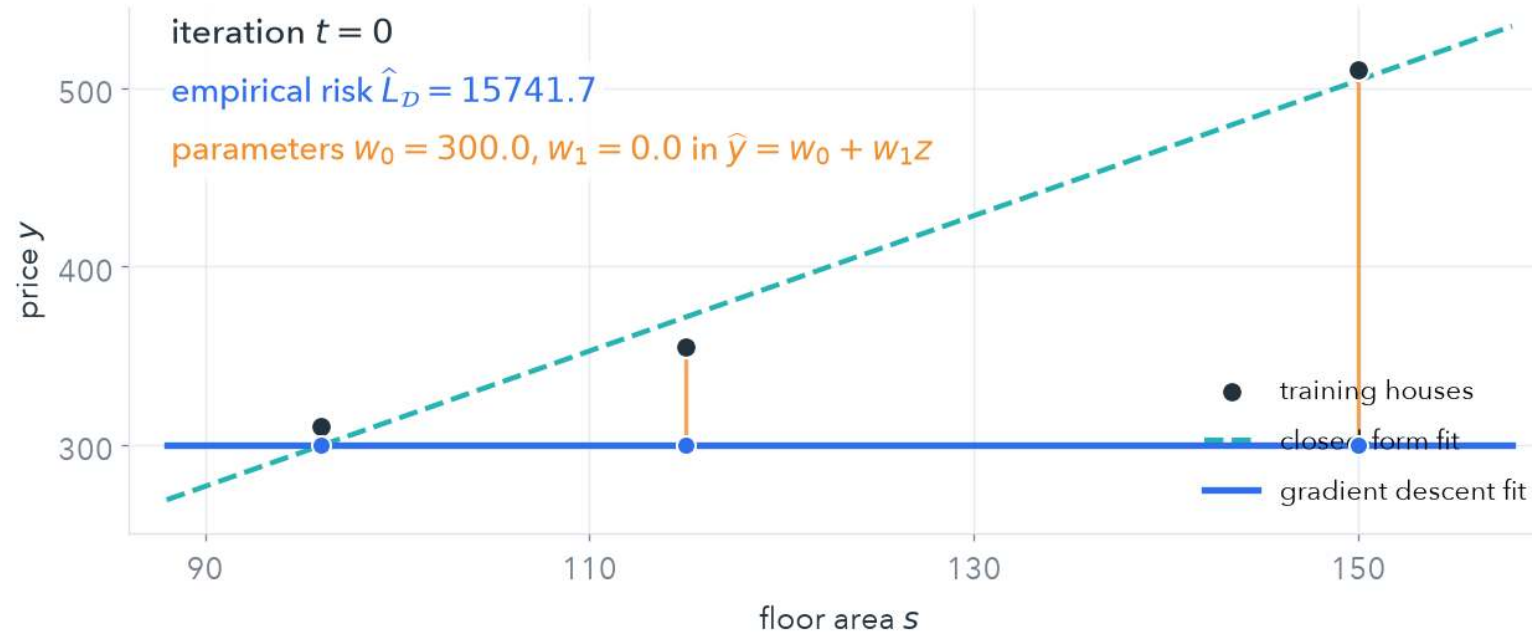
2. Repeat  $T$  times:

- compute  $\frac{\partial \hat{L}_{\mathcal{D}}}{\partial \mathbf{w}} \leftarrow \frac{2}{n} \sum_{i=1}^n (\mathbf{x}_i^{\top} \mathbf{w} - y_i) \mathbf{x}_i$
- update  $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial \hat{L}_{\mathcal{D}}}{\partial \mathbf{w}}$ .

3. Return  $\hat{f}(\mathbf{x}) = \mathbf{x}^{\top} \mathbf{w}$ .

 One update requires a contribution from every training example.

# Linear Regression Learns Iteratively



*i* This is batch gradient descent because every update uses all three training houses. The fitted line and empirical risk change after each full-batch update.

# What the Learner Returns

Gradient descent optimizes a vector, but the learner must return a predictor.

$$\mathcal{A}(\mathcal{D}) = \hat{f}, \quad \hat{f}(\mathbf{x}) = \mathbf{x}^\top \mathbf{w}^{(T)}.$$

## During Training

The algorithm updates the parameter vector  $\mathbf{w}$ .

## After Training

The final parameters define the predictor used on new inputs.

# The Cost of a Full Gradient

For each of  $n$  examples, linear regression computes

$$(\mathbf{x}_i^\top \mathbf{w} - y_i) \mathbf{x}_i.$$

If  $\mathbf{x}_i \in \mathbb{R}^d$ , one contribution requires work proportional to  $d$ .

cost of one full-data gradient =  $O(nd)$ .



If the dataset contains millions of examples, must we wait for all of them before taking every parameter step?

# Wrap-up

- The negative gradient gives a local descent direction.
- The learning rate controls the size of every update.
- Repeated updates produce an approximate minimizer.
- For several parameters, the gradient collects the partial derivatives.

 Next: use smaller groups of examples to update more frequently.